Enseignante : M.Karim Ghammam	Virtualization et Cloud **ORCHESTRATION ET KUBERNETES**	

Objectif : développer, conteneuriser, déployer et tester une application Web composée d'un frontend, d'une API REST Node.js et d'une base MySQL sur Kubernetes.

1. Objectifs pédagogiques

- Installer et utiliser Minikube et kubectl sur Ubuntu.
- Comprendre les Pods, Deployments, Services et PersistentVolumeClaims.
- Construire une image Docker pour une API Node.js et une interface Web.
- Déployer MySQL avec un stockage persistant et des identifiants gérés par Secret.
- Tester les communications entre les composants de l'application.
- Mettre à l'échelle les Deployments et diagnostiquer les erreurs courantes.

2. Scénario et architecture

Une université souhaite une application de gestion des étudiants permettant de consulter une liste et d'ajouter un étudiant. L'application comprend un frontend HTML/Nginx, un backend REST Node.js et une base de données MySQL.

```
Navigateur
    |
Service frontend (NodePort)
    |
Pods frontend (Nginx)
    |  /api/* via proxy
Service backend (ClusterIP)
    |
Pods backend (Node.js / Express)
    |
Service mysql (ClusterIP)
    |
MySQL + PVC (stockage persistant)
```

3. Environnement et prérequis

- VM Ubuntu 24.04 LTS sous VMware Workstation Pro.
- 2 à 4 vCPU, 8 Go de RAM recommandés et environ 25 Go d'espace disque libre.
- Connexion Internet et utilisateur disposant de sudo.
- Les parties de développement nécessitent Node.js/npm sur la VM.

Une configuration plus légère peut suffire pour les premières parties, mais MySQL, les builds et Kubernetes consomment des ressources supplémentaires.

Partie 1 — Installation de Docker et Minikube

1.1. Installer Minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

1.2. Installer kubectl

```
KUBECTL_VERSION=$(curl -L -s https://dl.k8s.io/release/stable.txt)
```

```
curl -LO "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
kubectl version --client
```

1.3. Démarrer le cluster

```
minikube start --driver=docker
kubectl get nodes
kubectl get pods -A
kubectl cluster-info
```

Exécuter Minikube avec l'utilisateur Ubuntu habituel, sans sudo. Si Docker renvoie une erreur de permission, reconnectez-vous après l'ajout au groupe docker ou utilisez temporairement sudo pour vérifier le daemon.

Questions

1. Quel est le rôle de Minikube ?
2. Quelle est la différence entre un conteneur et un Pod ?
3. Que signifie l'état Ready d'un nœud Kubernetes ?

Partie 2 — Premier déploiement Nginx

```
kubectl create deployment frontend-test --image=nginx
kubectl get deployments
kubectl get pods
kubectl expose deployment frontend-test --type=NodePort --port=80
kubectl get services
minikube service frontend-test --url
```

Ouvrir l'URL retournée dans le navigateur. Selon le réseau de la VM, l'accès peut nécessiter un transfert de port ou une autre configuration Minikube.

Questions

4. Quel est le rôle d'un Deployment ?
5. À quoi sert un Service ?
6. Quelle est la différence entre ClusterIP et NodePort ?

Partie 3 — Développement du backend Node.js

3.1. Préparer le projet

```
mkdir -p ~/gestion-etudiants/backend
cd ~/gestion-etudiants/backend
npm init -y
npm install express mysql2
```

3.2. Créer app.js

```
const express = require("express");
const mysql = require("mysql2/promise");
const app = express();
app.use(express.json());
```

```
const dbConfig = {
  host: process.env.DB_HOST || "mysql",
  user: process.env.DB_USER || "appuser",
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME || "universite"
};

app.get("/health", (req, res) => res.json({ status: "ok" }));

app.get("/students", async (req, res) => {
  try {
    const db = await mysql.createConnection(dbConfig);
    const [rows] = await db.execute("SELECT id, nom, email FROM etudiants");
    await db.end();
    res.json(rows);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: "Erreur de consultation" });
  }
});

app.post("/students", async (req, res) => {
  const { nom, email } = req.body;
  if (!nom || !email) return res.status(400).json({ error: "Nom et email obligatoires" });
  try {
    const db = await mysql.createConnection(dbConfig);
    const [result] = await db.execute(
      "INSERT INTO etudiants (nom, email) VALUES (?, ?)", [nom, email]
    );
    await db.end();
    res.status(201).json({ id: result.insertId, nom, email });
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: "Erreur d'ajout" });
  }
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, "0.0.0.0", () => console.log(`API sur ${PORT}`));
```

3.3. Créer le Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```

3.4. Construire et charger l'image

```
docker build -t gestion-etudiants-api:v1 .
minikube image load gestion-etudiants-api:v1
```

Questions

7. Quel est le rôle du Dockerfile ?
8. Quelle est la différence entre une image et un conteneur ?

9. Pourquoi l'application écoute-t-elle sur 0.0.0.0 ?
10. Quel est le rôle d'une API REST ?

Partie 4 — Déployer MySQL avec un stockage persistant

4.1. Créer un namespace et un Secret

```
kubectl create namespace gestion-etudiants

kubectl create secret generic mysql-secret \
  --namespace=gestion-etudiants \
  --from-literal=root-password='RootLab-2026!' \
  --from-literal=app-password='AppLab-2026!'
```

Les mots de passe ci-dessus sont réservés à ce laboratoire. Les Secrets Kubernetes ne sont pas nécessairement chiffrés dans etcd par défaut ; ne pas réutiliser ces valeurs en production.

4.2. Créer mysql.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: gestion-etudiants
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: gestion-etudiants
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: gestion-etudiants
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
```

```
    spec:
      containers:
        - name: mysql
          image: mysql:8.4
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: root-password
            - name: MYSQL_DATABASE
              value: universite
            - name: MYSQL_USER
              value: appuser
            - name: MYSQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: app-password
          volumeMounts:
            - name: mysql-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
```

```
kubectl apply -f mysql.yaml
kubectl get pods,svc,pvc -n gestion-etudiants
kubectl rollout status deployment/mysql -n gestion-etudiants --timeout=180s
```

4.3. Créer la table et insérer des données

```
kubectl exec -it deployment/mysql -n gestion-etudiants -- \
  sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -uroot universite'
```

Dans le client MySQL, exécuter les requêtes suivantes :

```
CREATE TABLE etudiants (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE
);

INSERT INTO etudiants (nom, email) VALUES
('Ahmed Ben Ali', 'ahmed@example.com'),
('Sarra Trabelsi', 'sarra@example.com'),
('Youssef Mansouri', 'youssef@example.com');

SELECT * FROM etudiants;
EXIT;
```

Questions

11. Expliquer la différence entre un PV et un PVC.
12. Pourquoi éviter d'exposer la base de données publiquement ?
13. Quel est le rôle du Service mysql ?

Partie 5 — Déployer le backend

5.1. Créer backend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: gestion-etudiants
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: gestion-etudiants-api:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 3000
          env:
            - name: PORT
              value: "3000"
            - name: DB_HOST
              value: mysql
            - name: DB_NAME
              value: universite
            - name: DB_USER
              value: appuser
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: app-password
---
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: gestion-etudiants
spec:
  selector:
    app: backend
  ports:
    - port: 3000
      targetPort: 3000
  type: ClusterIP
```

```
kubectl apply -f backend.yaml
kubectl get deployments,pods,services -n gestion-etudiants
kubectl rollout status deployment/backend -n gestion-etudiants
```

5.2. Tester l’API

```
kubectl port-forward -n gestion-etudiants service/backend 3000:3000
```

Dans un deuxième terminal :

```
curl http://127.0.0.1:3000/health
curl http://127.0.0.1:3000/students

curl -X POST http://127.0.0.1:3000/students \
  -H "Content-Type: application/json" \
  -d '{"nom":"Amira Gharbi","email":"amira@example.com"}'
```

Questions

14. Pourquoi le backend possède-t-il deux réplicas ?
15. Comment le backend trouve-t-il le Service MySQL ?
16. Pourquoi le Service backend est-il de type ClusterIP ?

Partie 6 — Développer et déployer le frontend

6.1. Créer le frontend

```
mkdir -p ~/gestion-etudiants/frontend
cd ~/gestion-etudiants/frontend
```

Créer le fichier index.html :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Gestion des étudiants</title>
</head>
<body>
  <h1>Gestion des étudiants</h1>
  <form id="student-form">
    <input id="nom" placeholder="Nom complet" required>
    <input id="email" type="email" placeholder="Adresse e-mail" required>
    <button type="submit">Ajouter</button>
  </form>
  <h2>Liste des étudiants</h2>
  <button id="refresh">Actualiser</button>
  <ul id="students"></ul>
  <script>
    const API = "/api";
    async function loadStudents() {
      const response = await fetch(`${API}/students`);
      const students = await response.json();
      const list = document.getElementById("students");
      list.replaceChildren();
      students.forEach(student => {
        const item = document.createElement("li");
        item.textContent = `${student.nom} — ${student.email}`;
        list.appendChild(item);
      });
    }
    document.getElementById("refresh").addEventListener("click", loadStudents);
    document.getElementById("student-form").addEventListener("submit", async event => {
      event.preventDefault();
```

```
      const nom = document.getElementById("nom").value;
      const email = document.getElementById("email").value;
      const response = await fetch(`${API}/students`, {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({nom, email})
      });
      if (!response.ok) {
        alert("Erreur lors de l'ajout");
        return;
      }
      event.target.reset();
      await loadStudents();
    });
    loadStudents();
  </script>
</body>
</html>
```

6.2. Créer default.conf

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://backend:3000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

6.3. Créer le Dockerfile frontend

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
COPY default.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

6.4. Construire et charger l'image

```
docker build -t gestion-etudiants-frontend:v1 .
minikube image load gestion-etudiants-frontend:v1
```

6.5. Créer frontend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: gestion-etudiants
spec:
  replicas: 2
```

```
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: gestion-etudiants-frontend:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: gestion-etudiants
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
  type: NodePort
```

```
kubectl apply -f frontend.yaml
kubectl get pods,services -n gestion-etudiants
kubectl rollout status deployment/frontend -n gestion-etudiants
minikube service frontend -n gestion-etudiants --url
```

Ouvrir l'URL retournée dans le navigateur. Le frontend utilise le proxy Nginx pour transmettre /api/students au backend. Vérifier l'affichage de la liste et l'ajout d'un étudiant.

Partie 7 — Scaling et remplacement automatique

```
kubectl scale deployment backend --replicas=4 -n gestion-etudiants
kubectl scale deployment frontend --replicas=3 -n gestion-etudiants
kubectl get deployments,pods -o wide -n gestion-etudiants
```

Identifier un Pod backend, le supprimer puis observer le remplacement automatique :

```
kubectl get pods -n gestion-etudiants
kubectl delete pod NOM_DU_POD -n gestion-etudiants
kubectl get pods -n gestion-etudiants
```

Questions

17. Définir le scaling horizontal.
18. Pourquoi Kubernetes recrée-t-il le Pod supprimé ?
19. Pourquoi plusieurs réplicas ne garantissent-ils pas à eux seuls une haute disponibilité complète ?

Partie 8 — Diagnostic

```
kubectl get all -n gestion-etudiants
```

```
kubectl logs deployment/backend -n gestion-etudiants
kubectl describe pod NOM_DU_POD -n gestion-etudiants
kubectl get events -n gestion-etudiants --sort-by=.metadata.creationTimestamp
```

Exercice : modifier temporairement DB_HOST dans backend.yaml pour utiliser un nom de service inexistant. Redéployer, examiner les logs, identifier le problème puis rétablir DB_HOST=mysql.

Questions

20. Que signifie CrashLoopBackOff ?
21. Comment diagnostiquer un problème de connexion à MySQL ?
22. Quelle est la différence entre kubectl logs et kubectl describe ?

Partie 9 — Travail supplémentaire

- Ajouter une fonctionnalité de suppression d'étudiant.
- Ajouter une route de consultation par identifiant.
- Remplacer le frontend HTML par une interface React.
- Ajouter des readinessProbe et livenessProbe.
- Définir des requests et limits CPU/mémoire.
- Installer Prometheus et Grafana pour surveiller le cluster.

10. Nettoyage

```
kubectl delete namespace gestion-etudiants
kubectl delete deployment frontend-test --ignore-not-found
kubectl delete service frontend-test --ignore-not-found
minikube stop
```

La suppression du namespace efface les ressources qu'il contient. Vérifier les données à conserver avant le nettoyage. Pour supprimer complètement le cluster de laboratoire, exécuter minikube delete.

Conclusion générale

Ce TP met en pratique le développement et le déploiement d'une application Web sur Kubernetes : construction d'images Docker, API REST Node.js, base MySQL, stockage persistant, Services, scaling et diagnostic. Il constitue une introduction concrète aux architectures distribuées et aux pratiques DevOps.